



Lecture 3

Problem Solving as Search & State Space Search

1. Solving Problems by Searching

- Types of Agents (Reflex vs. Planning) - State Space Search - Example: Tic-Tac-Toe Game - Example: Mechanical Fault Diagnosing - How human beings think? - Heuristic Search

3.2 State Space Search Graph & Strategies

- Search problem components - State Space & State Space Graph - Examples: Vacuum World, The 8-Puzzle, Romania, etc. - State Space Search Strategies - Selecting Search Strategy

3.3 Basic Idea of Search & the Backtracking Search Algorithm

- Search: Basic idea - Search Tree - Tree Search Algorithm - Outline & Example - Handling Repeated States - Backtracking Search Algorithm & Data Structures

Where are we now .. ?!

- 1: Course Introduction & Plan
- 2: An introduction to Artificial Intelligence [AI]
- 3: Intelligent Agents, & AI Related Disciplines
- 4: Solving Problems by Searching, and State Space Search Strategies & Structures
- 5: Knowledge Representation via Propositional & Predicate Calculi
- 6: Problem Solving as Search (Blind / Uninformed vs. Heuristic / Informed Strategies)
- 7: Problem Solving as Search (More on Heuristic Search + Adversarial Search)
- 8: Beyond Classical Search (Evolutionary / Genetic Algorithms)
- 9: Heuristic Optimization, Hill Climbing, Gradient Descent, & Simulated Annealing
- 10: Supervised Machine Learning: Decision Trees via the ID3 Algorithm + Business Intelligence app. via Weka
- 11: The Learning Problem, the Perceptron, & the Perceptron Learning Algorithm [PLA]
- 12: Multilayer Perceptron [MLP], & an introduction to Artificial Neural Networks [ANNs]
- 13: Loss Functions, Weights Optimization, & Support Vector Machines [SVMs]

Course & Lectures are based on their counterparts in the following:

- Intelligent Systems, University of British Columbia (*Dept. of CS*)
- Introduction to Artificial Intelligence, University of Wisconsin-Madison
- Artificial Intelligence, University of Illinois at Urbana-Champaign
- Artificial Intelligence, University of California, Berkeley
- S. Russell & P. Norvig, "Artificial Intelligence: A Modern Approach," *3rd Ed*
- G. Luger, "Artificial Intelligence: Structures & Strategies for Complex Problem Solving," *5th Ed*
- W. Ertel, "Introduction to Artificial Intelligence," *2nd Ed*

Lecture 3: Solving Problem by Searching, .. & .. State Space Search Strategies and Structures

1. Solving problems by Searching

- Solving Problems by Searching
- Types of Agents (Reflex vs. Planning)
- State Space Search
- Example: Tic-Tac-Toe Game
- Example: Mechanical Fault Diagnosing
- How human beings think?
- Heuristic Search

3.2 State Space Search Graph & Strategies

- Search problem components
- Example: Romania
- State Space & State Space Graph
- Example: Vacuum World
- Example: The 8-Puzzle
- Example: Robot Motion Planning

- Example: Tic-Tac-Toe
- Example: Traveling Salesperson
- State Space Search Strategies
- Selecting Search Strategy

3.3 Basic Idea of Search & the Backtracking Search Algorithm

- Search: Basic idea
- Search Tree
- Tree Search Algorithm Outline
- Tree Search Example
- Handling repeated states
- Backtracking Search
- Backtracking Algorithm Data Structures
- Backtracking Algorithm

3.4 Practice Exercises (Solved Problems)

Resources for this lecture

- This lecture covers the following chapters:
 - **Chapter 3 (Solving Problems by Searching; only sections 3.1, 3.2, & 3.3)** from Stuart J. Russell and Peter Norvig, Artificial Intelligence: A Modern Approach (Third Edition, 2010), by Pearson Education Inc.

.. AND ..

- **Part II (pages 41 to 45) and Chapter 3 (Structures and Strategies for State Space Search; only sections 3.0, 3.1, and 3.2)** from George F. Luger, Artificial Intelligence: Structures and strategies for complex problem solving, Fifth Edition (2005), Pearson Education Limited.

Solving Problems by SEARCHING



Types of Agents

α Reflex Agent :



Considers how the world IS

- Choose action based on current percept.
- Do not consider the future consequences of actions.

α Planning agent :



Considers how the world WOULD BE

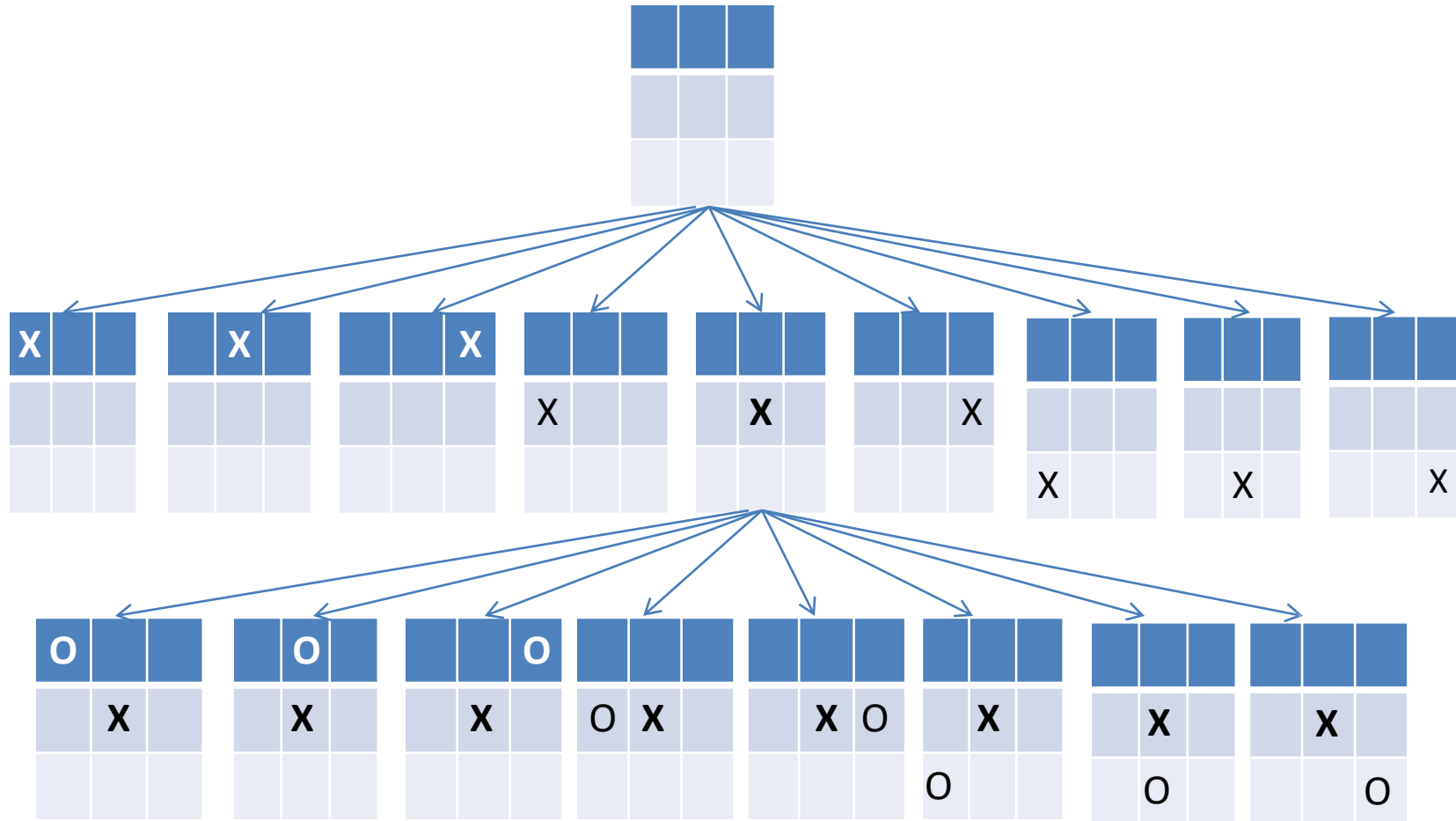
- Decisions based on (hypothesized) consequences of actions.
- Must have a model of how the world evolves in response to actions.
- Must formulate a goal.

StateSpaceSearch

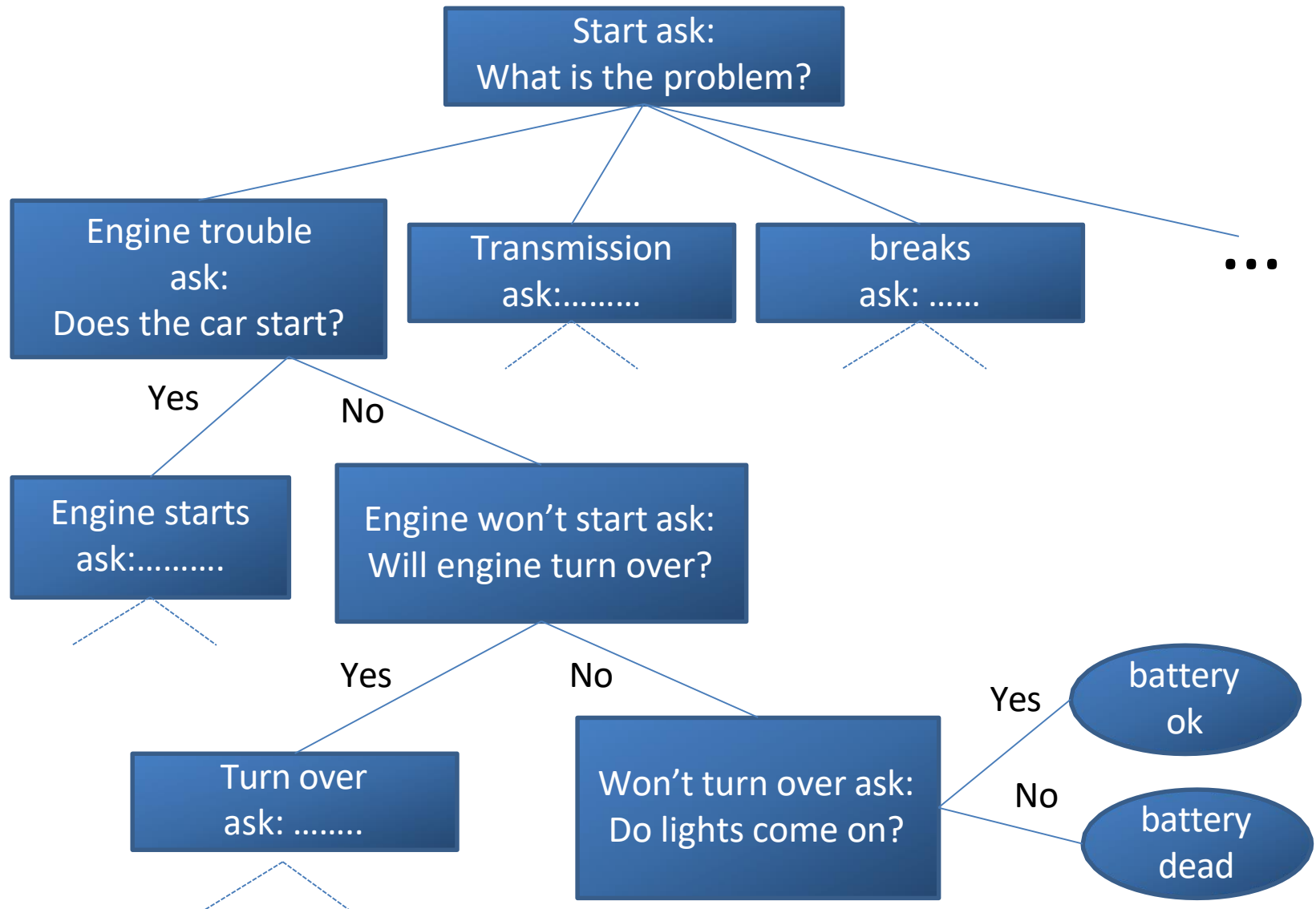
Problems are solved by searching among alternative choices.

- Humans consider several alternative strategies on their way to solving a problem.
 - A *Chess* player considers a few alternative moves.
 - A *mathematician* chooses from a different strategies to find a proof for a theorem.
 - A *physician* evaluates several possible diagnoses.

Example: **Tic-Tac-Toe** Game



Example: Mechanical Fault Diagnosing



How Human Beings Think..?

- Human beings do not search the entire state space (*exhaustive search*).
- Only alternatives that experience has shown to be effective are explored.
- Human problem solving is based on judgmental rules that limit the exploration of search space to those portions of state space that seem somehow promising.
- These judgmental rules are known as “*heuristics*”.

Heuristic Search

- A heuristic is a strategy for selectively exploring the search space.
- It guides the search along lines that have a high probability of success.
- It employs knowledge about the nature of a problem to find a solution.
- It does not guarantee an optimal solution to the problem but can come close most of the time.
- Human beings use many heuristics in problem solving.

Exercises

What have we learned?

- Define the following terms:
 - State Space Graph,
 - Exhaustive Search, and ..
 - Heuristics.
- Describe briefly the difference between a Planning Agent & a Reflex Agent.
- Give examples of Heuristics that human beings employ in any domain.

Additional Resources

Up Next ..
Section 3.2

- Searching:

<https://www.youtube.com/watch?v=t0yCDZe6Tnk>

- State Space Problems:

<https://www.youtube.com/watch?v=ngkXAcjeNWE>

Lecture 3: Solving Problem by Searching, .. & .. State Space Search Strategies and Structures

1. Solving problems by Searching

- Solving Problems by Searching
- Types of Agents (Reflex vs. Planning)
- State Space Search
- Example: Tic-Tac-Toe Game
- Example: Mechanical Fault Diagnosing
- How Human Beings Think?
- Heuristic Search

3.2 State Space Search Graph & Strategies

- Search Problem Components
- Example: Romania
- State Space & State Space Graph
- Example: Vacuum World
- Example: The 8-Puzzle
- Example: Robot Motion Planning

■ Example: Tic-Tac-Toe

- Example: Traveling Salesperson
- State Space Search Strategies
- Selecting Search Strategy

3.3 Basic Idea of Search & the Backtracking Search Algorithm

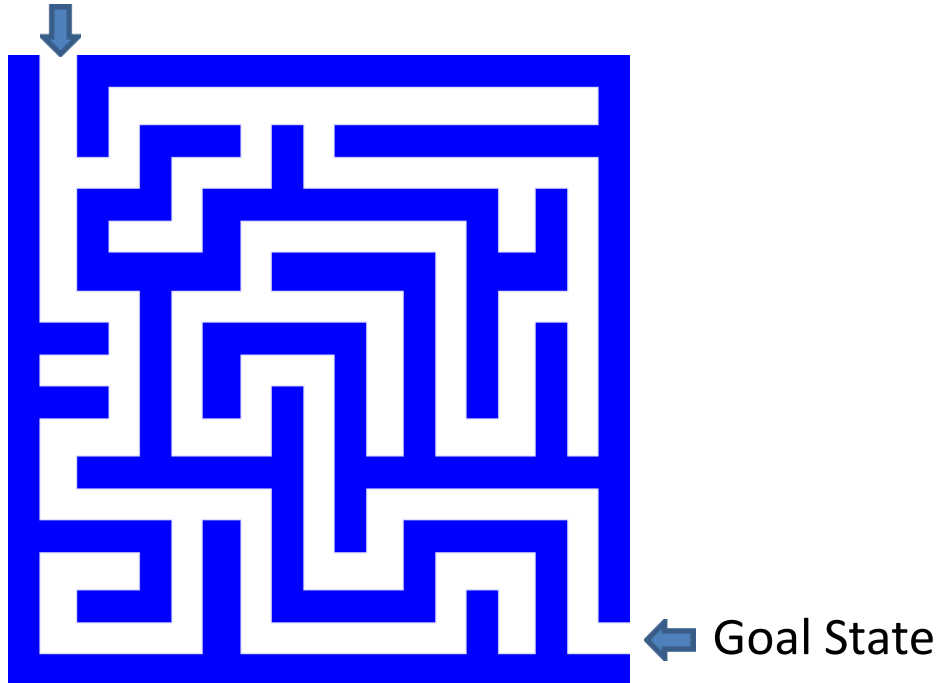
- Search: Basic idea
- Search Tree
- Tree Search Algorithm Outline
- Tree Search Example
- Handling repeated states
- Backtracking Search
- Backtracking Algorithm Data Structures
- Backtracking Algorithm

3.4 Practice Exercises (Solved Problems)

Search

We will consider the problem of designing **goal-based agents** in **fully observable, deterministic, discrete, known** environments.

Start State



Search

We will consider the problem of designing **goal-based agents** in **fully observable, deterministic, discrete, known** environments.

The agent must find a *sequence of actions* that reaches the goal. The **performance measure** is defined by:

- (a) reaching the goal, *and ..*
- (b) how “expensive” the path to the goal is.

Search Problem Components

Initial state

Actions

Transition model

What state results from performing a given action in each state?

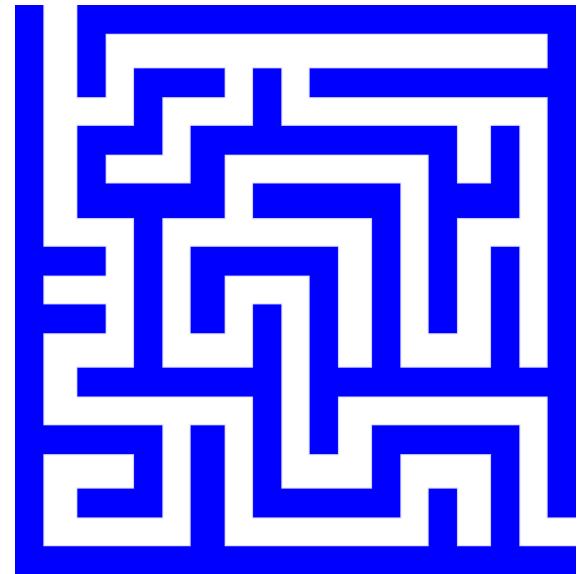
Goal state

Solution Path

Path cost

Assume that it is a sum of nonnegative *step costs*

Initial
State



Goal
State



The **optimal solution** is the sequence of actions that gives the lowest path cost for reaching the goal.

Example: Romania

- On vacation in Romania; currently in Arad.
- Flight leaves tomorrow from Bucharest.



Initial state

- Arad

Actions

- Go from one city to another

Transition Model

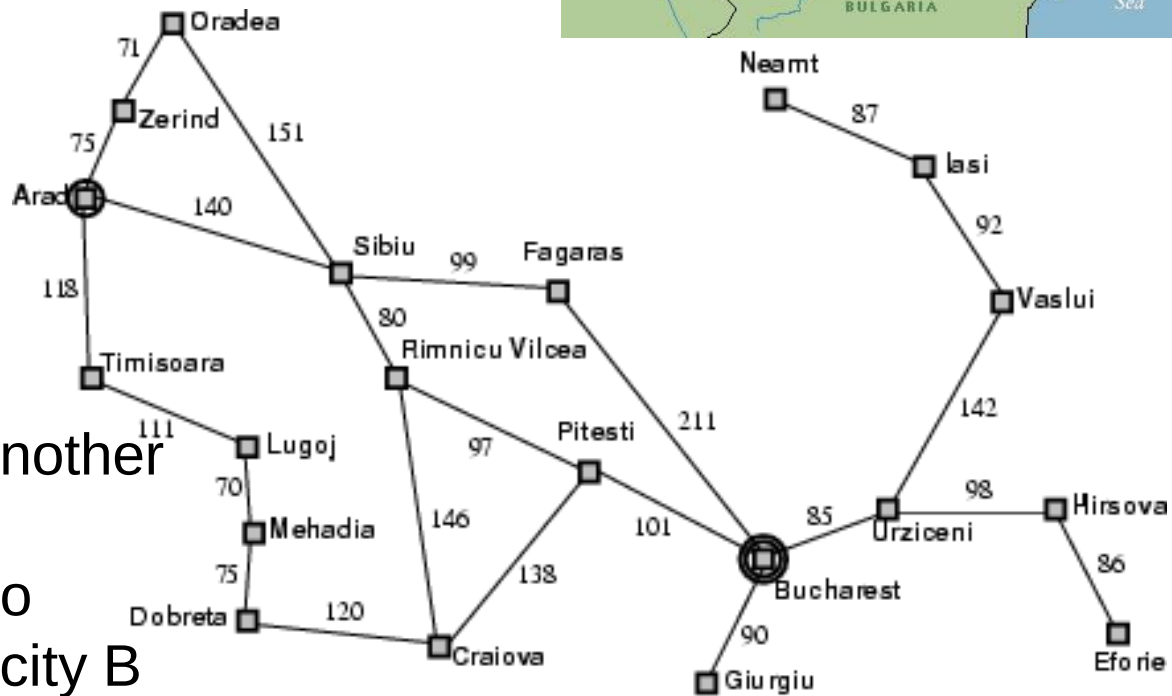
- If you go from city A to city B, you end up in city B

Goal State

- Bucharest

Path Cost

- Sum of edge costs (*total distance traveled*)

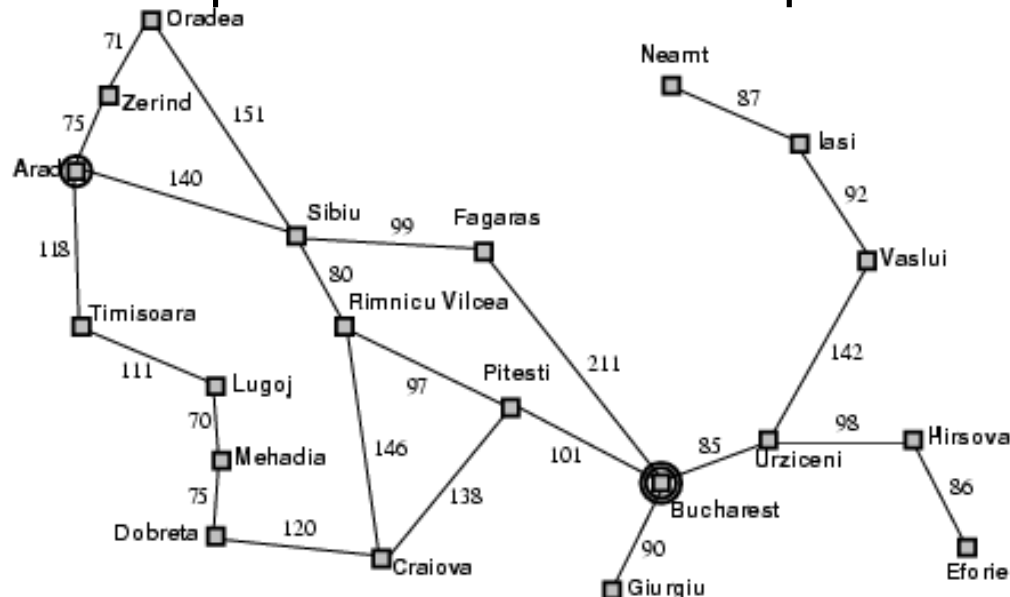


StateSpace

The initial state, actions, and transition model define the **state space** of the problem;

- The set of all states reachable from initial state by any sequence of actions.
- Can be represented as a **directed graph** where the nodes are states and links between nodes are actions.

What is the state space for the Romania problem?



StateSpace

An AI problem can be represented as a *state space graph*.

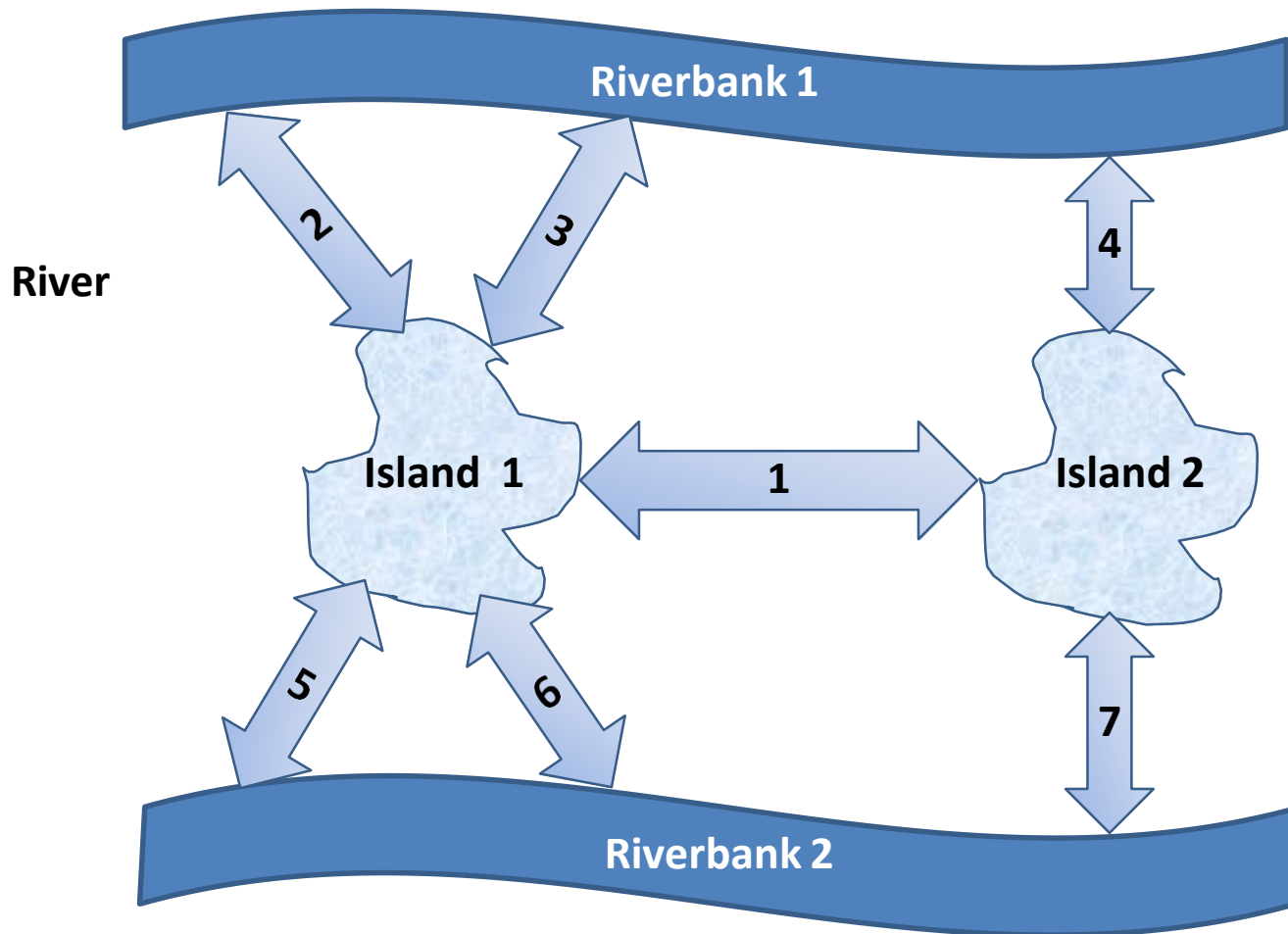
A **graph** is a set of *nodes* and *links* that connect them.

Graph theory:

- Labeled graph.
- Directed graph.
- Path.
- Rooted graph.
- Tree.
- Parent.
- Child.
- Sibling.
- Ancestor.
- Descendant.

StateSpace

*Graph Theory, Königsberg Bridges Problem,
& Euler Tour ..*

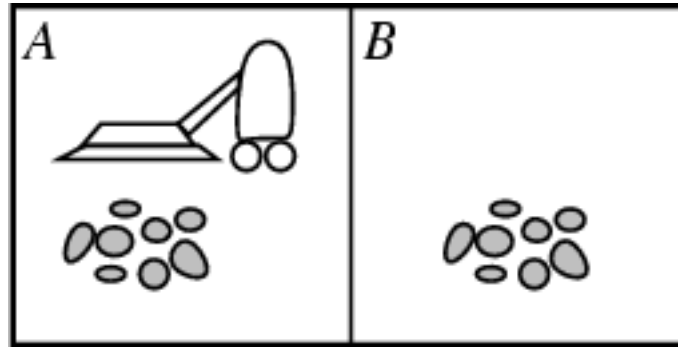


StateSpace

A state space is represented by four-tuple $[N, A, S, GD]$.

- **N**, is the set of nodes or states of the graph. These correspond to the states in the problem-solving process.
- **A**, is the set of arcs between nodes. These correspond to the steps in a problem-solving process.
- **S**, a nonempty subset of N , contains the start-state (s) of the problem.
- **GD**, a nonempty subset of N , contains the goal-state(s) of the problem. The states in GD are described using either:
 - A measurable property of the states encountered in the search.
 - A property of the solution path developed in the search (*a **solution path** is a path through this graph from a node in S to a node in GD .*)

Example: Vacuum World



- **States:**

- Agent location and dirt location
- How many possible states?
- What if there are n possible locations?
 - The size of the state space grows exponentially with the “size” of the world!

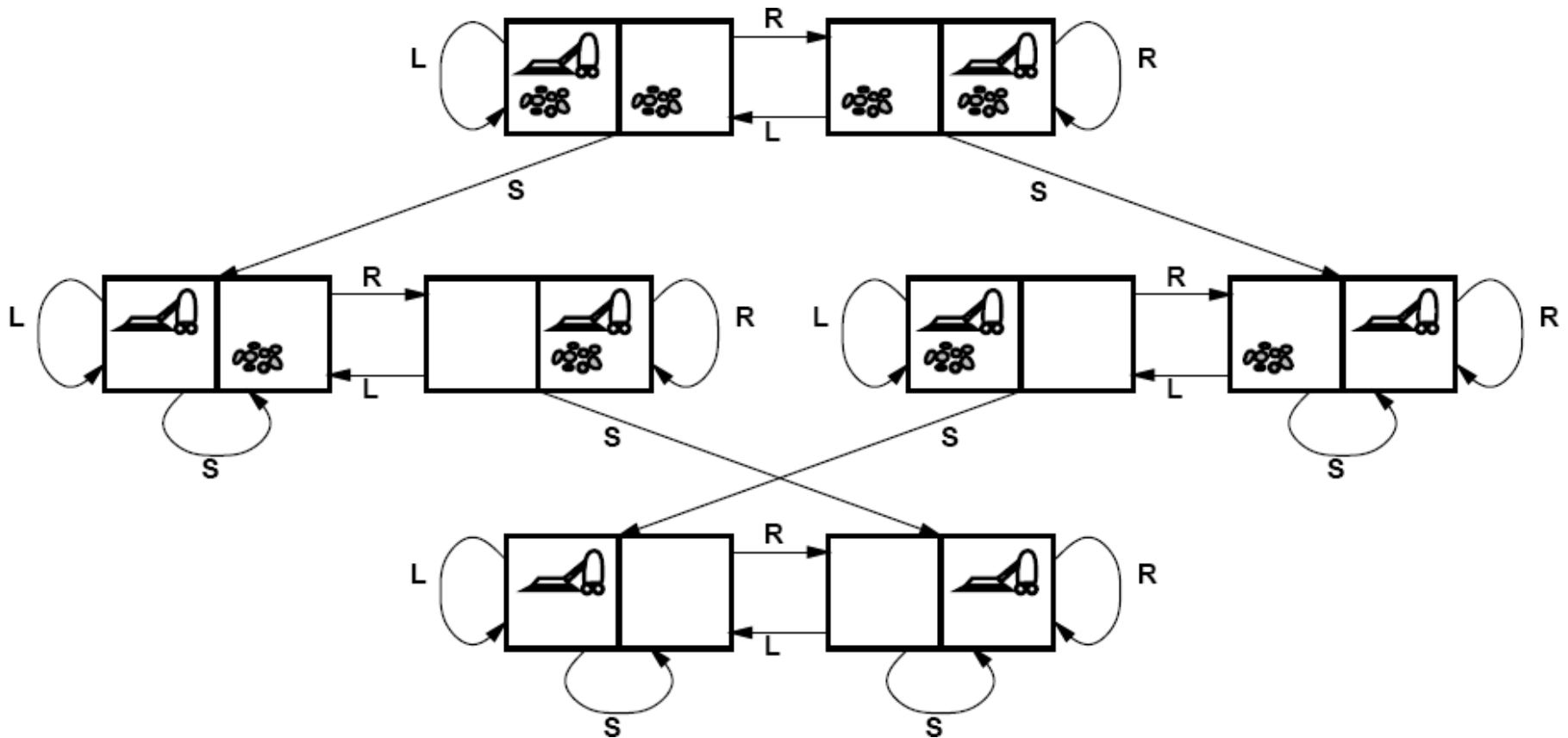
- **Actions:**

- Left, right, suck.

- **Transition Model .. ?**

Example: Vacuum World State Space Graph

○ Transition Model:



Example: the 8-Puzzle

- **States**

- Locations of tiles
 - 8-puzzle: 181,440 states ($9!/2$)
 - 15-puzzle: ~10 trillion states
 - 24-puzzle: $\sim 10^{25}$ states

- **Actions**

- Move blank left, right, up, down

- **Path Cost**

- 1 per move

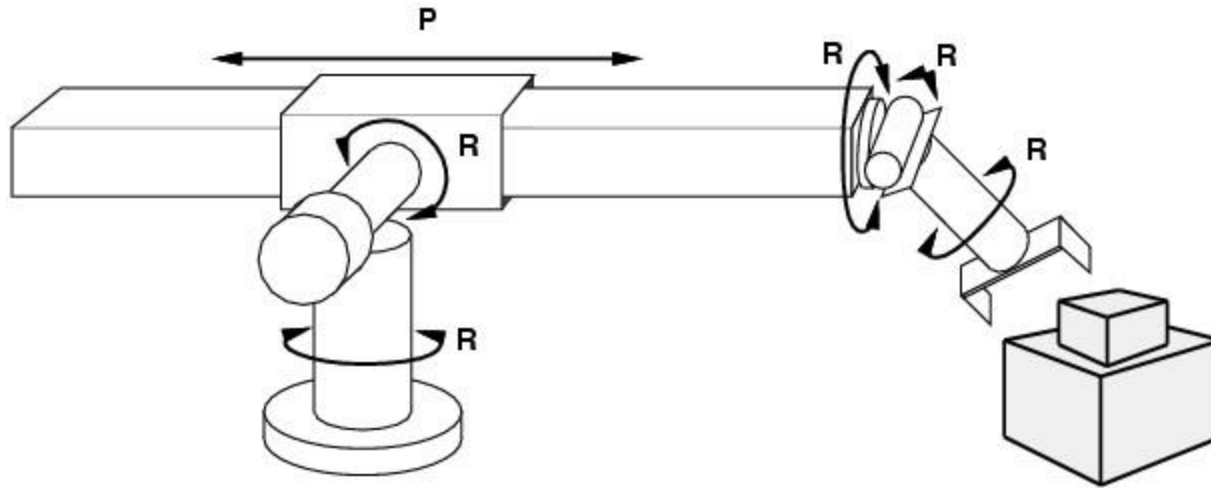
7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Example: Robot Motion Planning



- **States**
 - Real-valued joint parameters (*angles, displacements*).
- **Actions**
 - Continuous motions of robot joints.
- **Goal State**
 - Configuration in which object is grasped.
- **Path Cost**
 - Time to execute, smoothness of path, etc.

Example: Tic-Tac-Toe

- **Nodes(N)**: all the different configuration of Xs and Os that the game can have.
- **Arcs (A)**: generated by legal moves by placing an X or an O in unused location.
- **Start state (S)**: an empty board.
- **Goal states (GD)**: a board state having three Xs in a row, column, or diagonal.
- The arcs are directed, then no cycles in the state space, directed acyclic graph (DAG).
- **Complexity**: 9! Different paths can be generated.

Example: **Traveling Salesperson**

A salesperson has five cities to visit and then must return home.

- **Nodes(N)**: represent 5 cities.
- **Arcs(A)**: labeled with weight indicating the cost of traveling between connected cities.
- **Start state(S)**: a home city.
- **Goal states(GD)**: an entire path contains a complete circuit with minimum cost.
- **Complexity**: $(n-1)!$ Different cost-weighted paths can be generated.

State Space Search Strategies

There are two distinct ways for searching a state space graph:

- **Data-Driven Search: (Forward chaining)**

Start searching from the given data of a problem instance toward a goal.

- **Goal-Driven Search: (Backward chaining)**

Start searching from a goal state to facts or data of the given problem.

State Space Search Strategies ..

Selecting Search Strategy

Data-Driven Search is suggested if:

- The data are given in the initial problem statement.
- There are few ways to use the given facts.
- There are large number of potential goals.
- It is difficult to form a goal or hypothesis.

Goal- Driven Search is appropriate if:

- A goal is given in the problem statement or can easily formulated.
- There are large number of rules to produce a new facts.
- Problem data are not given but acquired by the problem solver.

Exercises

What have we learned?

- Define the following terms: Path, Rooted Graph, Tree.
- Describe using drawing the Kongsberg problem "Euler Tour."
- Determine whether goal-driven or data-driven search would be preferable for solving each of the following problems. Justify your answer.
 - You have met a person who claims to be your distant cousin, with a common ancestor named John Doe. You would like to verify her claim.
 - Another person claims to be your distant cousin. He doesn't know the common ancestor's name but knows that it was no more than eight generations back. You would like to either find this ancestor or determine that she didn't exist.
 - A theorem prover for plane geometry.

Additional Resources

Up Next ..
Section 3.3

- Backward & Forward Chaining

<https://www.youtube.com/watch?v=ZhTt-GG7PiQ>

Lecture 3: Solving Problem by Searching, .. & .. State Space Search Strategies and Structures

1. Solving problems by Searching

- Solving Problems by Searching
- Types of Agents (Reflex vs. Planning)
- State Space Search
- Example: Tic-Tac-Toe Game
- Example: Mechanical Fault Diagnosing
- How human beings think?
- Heuristic Search

3.2 State Space Search Graph & Strategies

- Search problem components
- Example: Romania
- State Space & State Space Graph
- Example: Vacuum World
- Example: The 8-Puzzle
- Example: Robot Motion Planning

- Example: Tic-Tac-Toe
- Example: Traveling Salesperson
- State Space Search Strategies
- Selecting Search Strategy

3.3 Basic Idea of Search & the Backtracking Search Algorithm

- Search: Basic idea
- Search Tree
- Tree Search Algorithm Outline
- Tree Search Example
- Handling repeated states
- Backtracking Search
- Backtracking Algorithm Data Structures
- Backtracking Algorithm

3.4 Practice Exercises (Solved Problems)

Search..?

Given:

- ✓ **Initial state**
- ✓ **Actions**
- ✓ **Transition model**
- ✓ **Goal state**
- ✓ **Path cost**

How do we find the optimal solution?

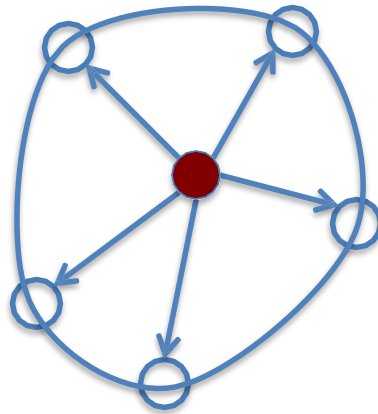
Search: Basic idea

- Let's begin at the start state and **expand** it by making a list of all possible successor states.
- Maintain a **frontier** or a list of unexpanded states.
- At each step, pick a state from the frontier to expand.
- Keep going until you reach a goal state.
- Try to expand as few states as possible.

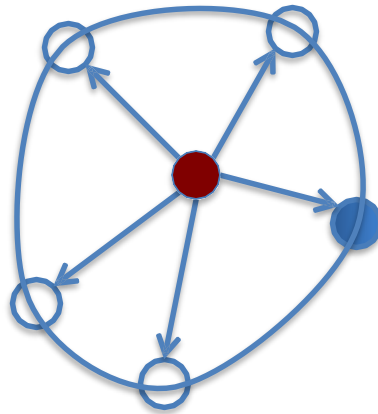
Search: Basic idea



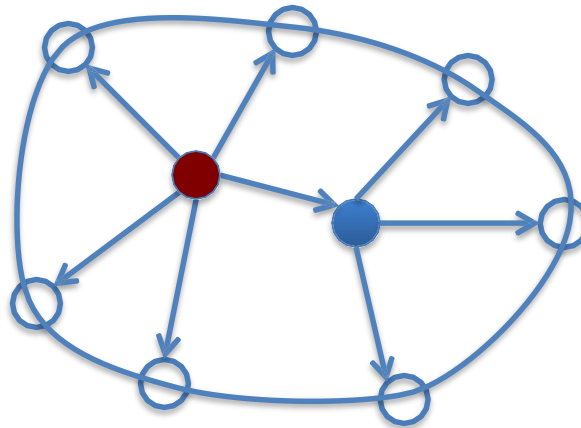
Search: Basic idea



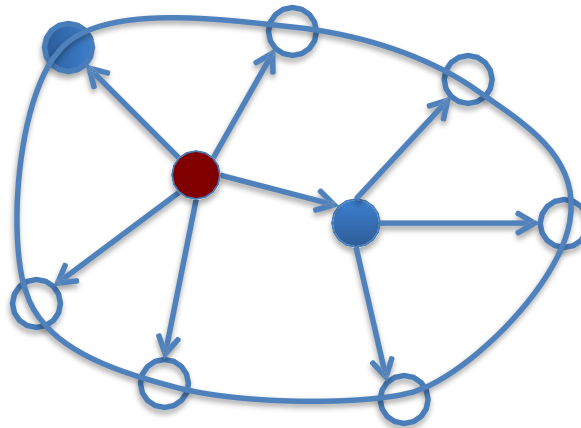
Search: Basic idea



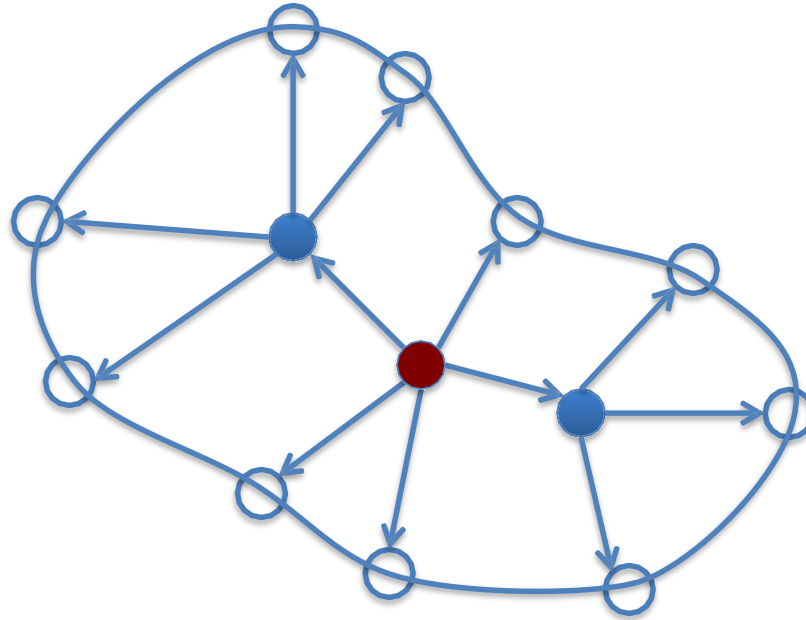
Search: Basic idea



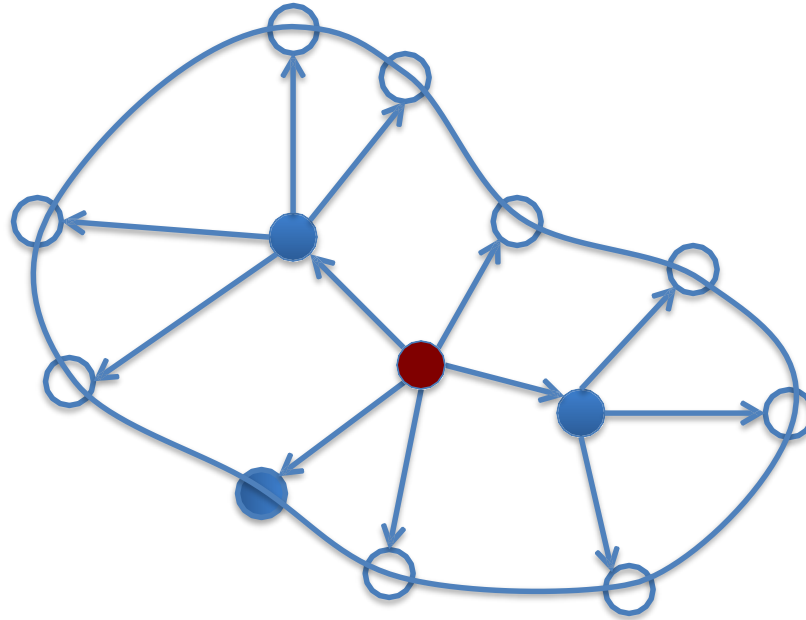
Search: Basic idea



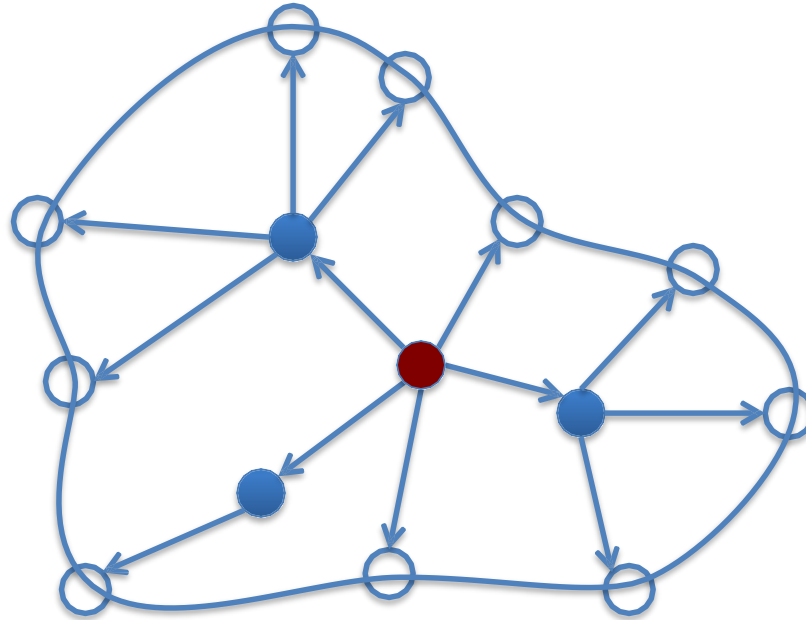
Search: Basic idea



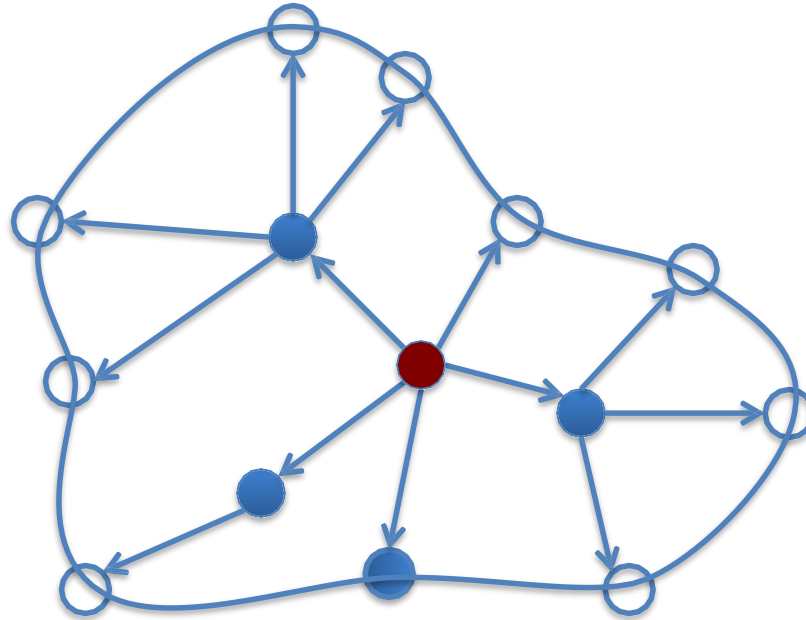
Search: Basic idea



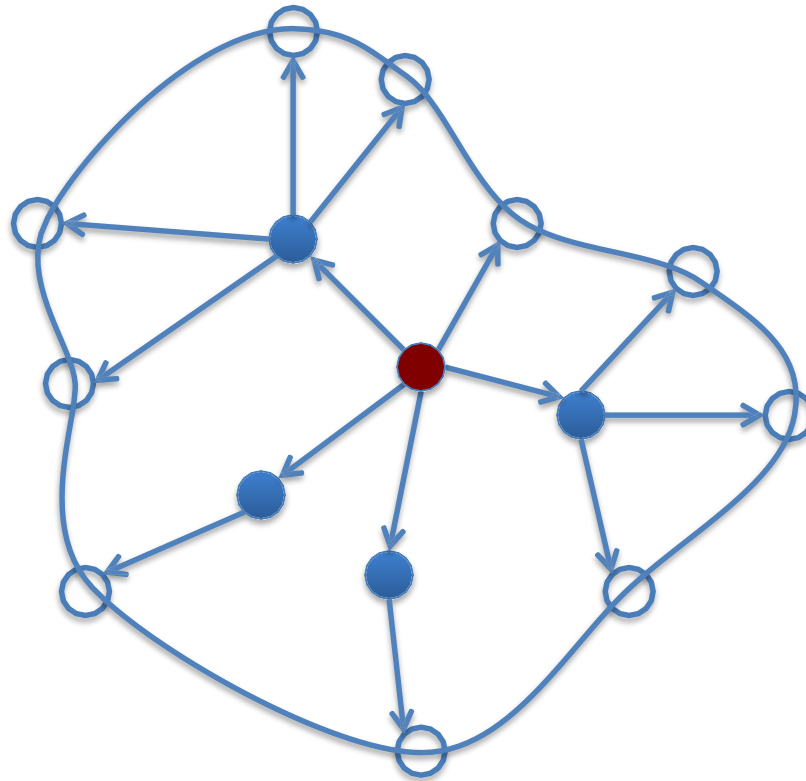
Search: Basic idea



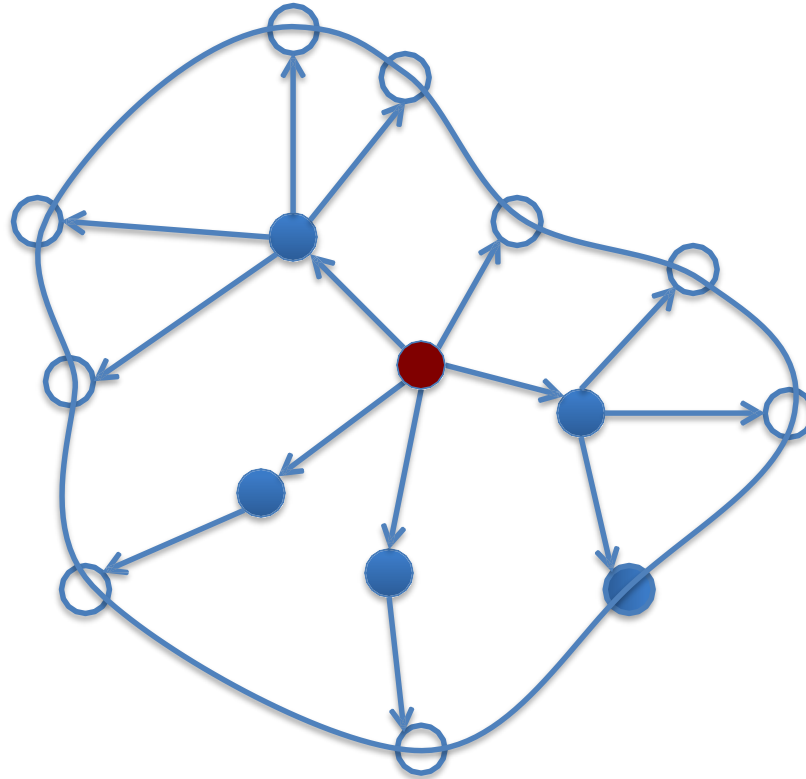
Search: Basic idea



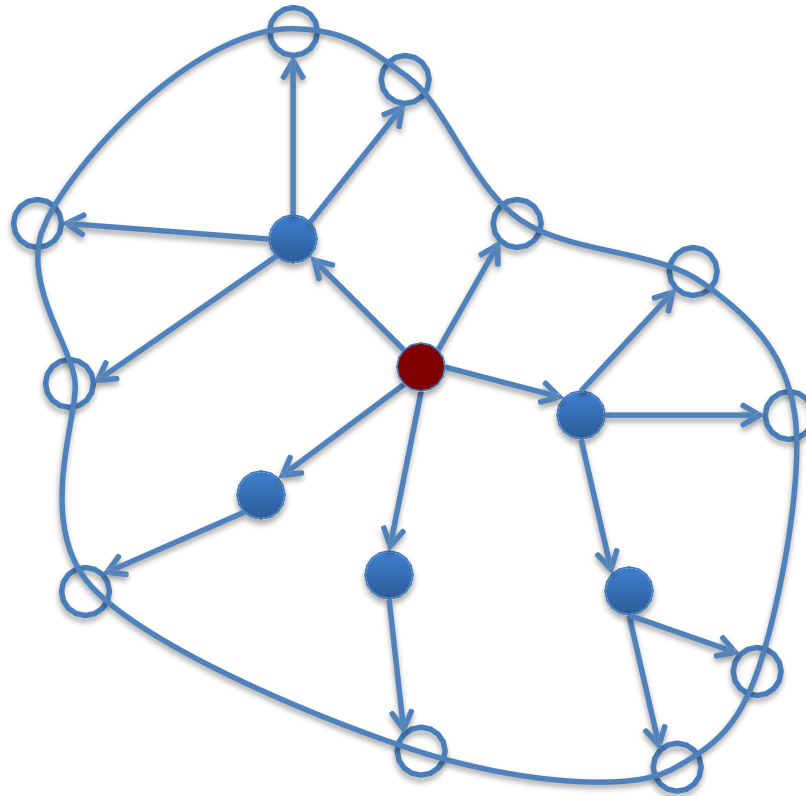
Search: Basic idea



Search: Basic idea



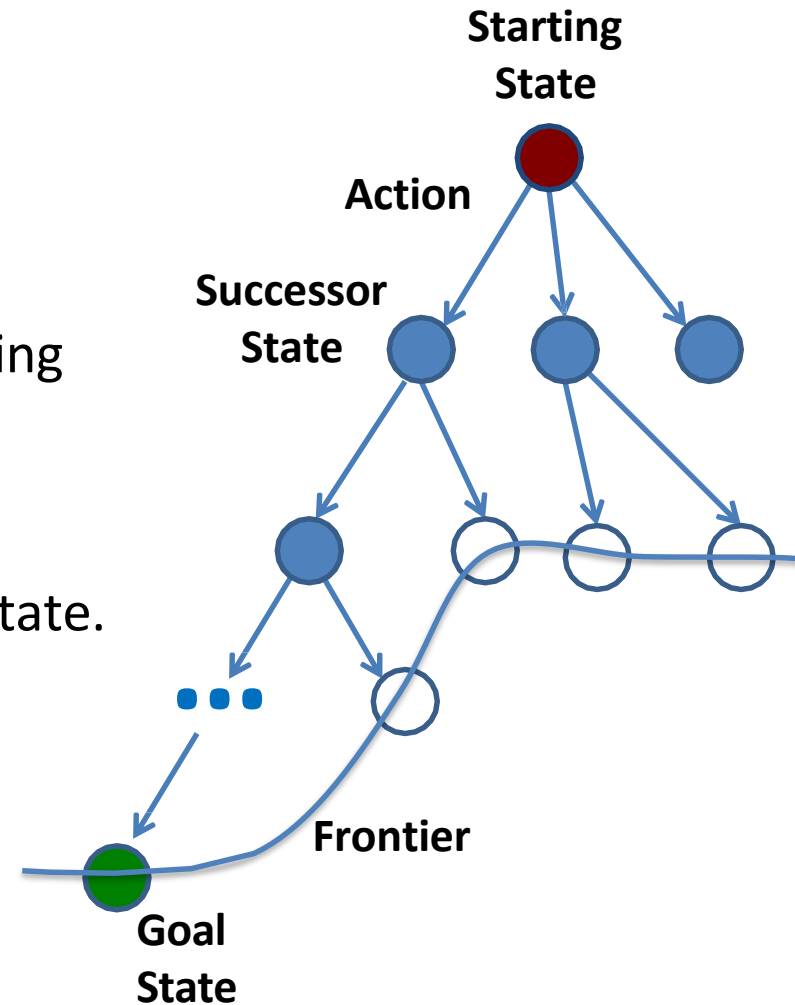
Search: Basic idea



Search Tree (the What-if tree)

“What if” tree of sequences of actions and outcomes;

- I.e., When we are searching, we are not acting in the world, merely “thinking” about the possibilities.
- The root node corresponds to the starting state.
- The children of a node correspond to the successor states of that node’s state.
- A path through the tree corresponds to a sequence of actions.
- A solution is a path ending in the goal state



Nodes vs. States ..? A state is a representation of the world, while a node is a data structure that is part of the search tree. Node must keep pointer to parent, path cost, possibly other info.

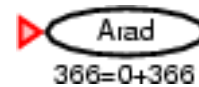
Tree Search Algorithm Outline

Initialize the **frontier** using the **starting state**.

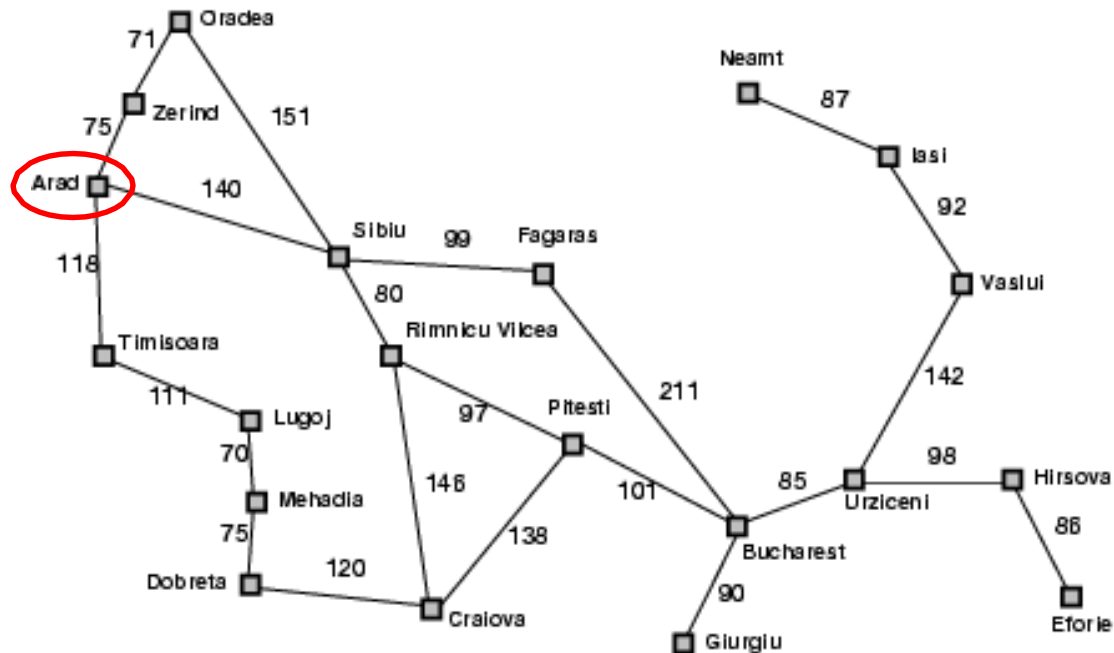
While the frontier is not empty:

- Choose a frontier node according to **search strategy** and take it off the frontier.
- If the node contains the **goal state**, return solution.
- Else **expand** the node and add its children to the frontier.

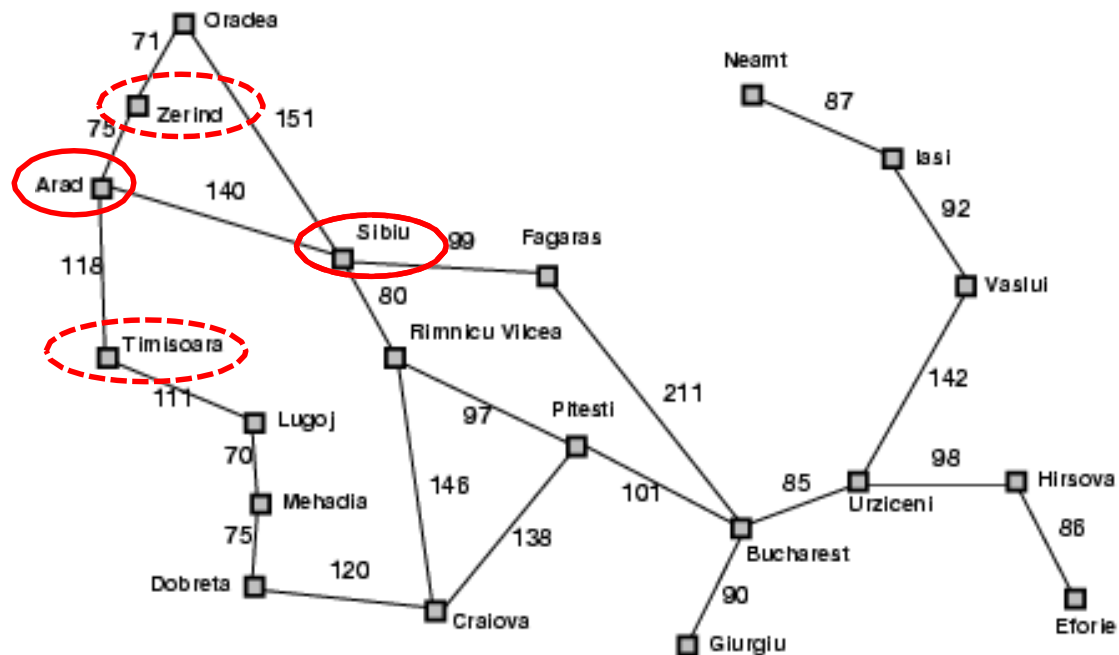
Tree Search Example



Start: Arad
Goal: Bucharest

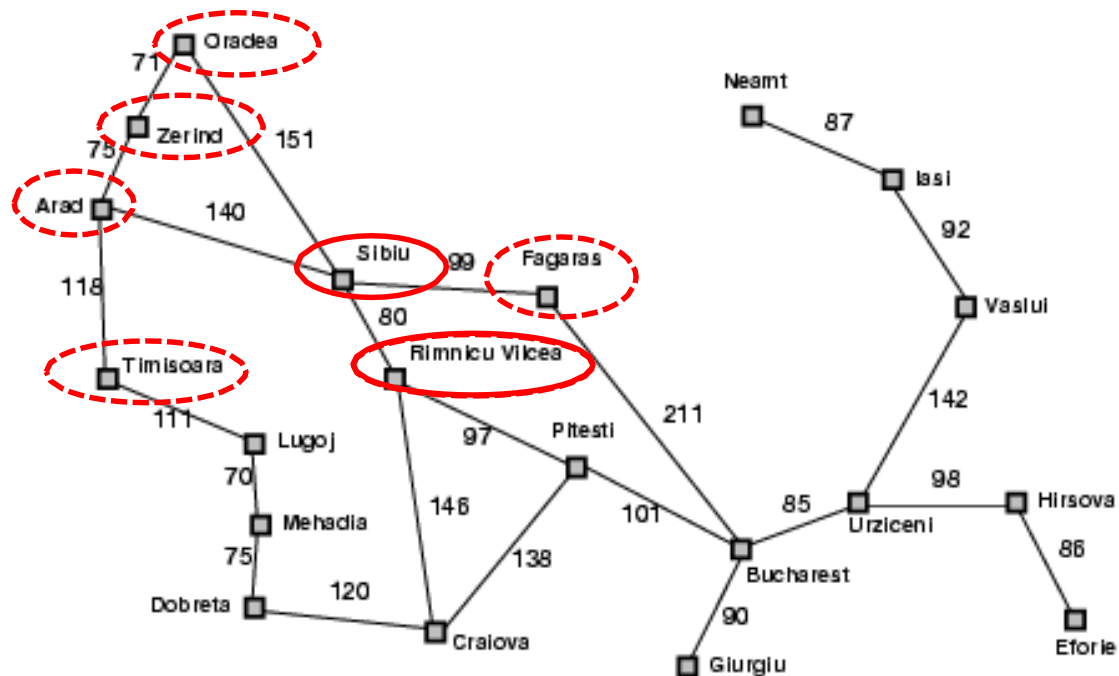
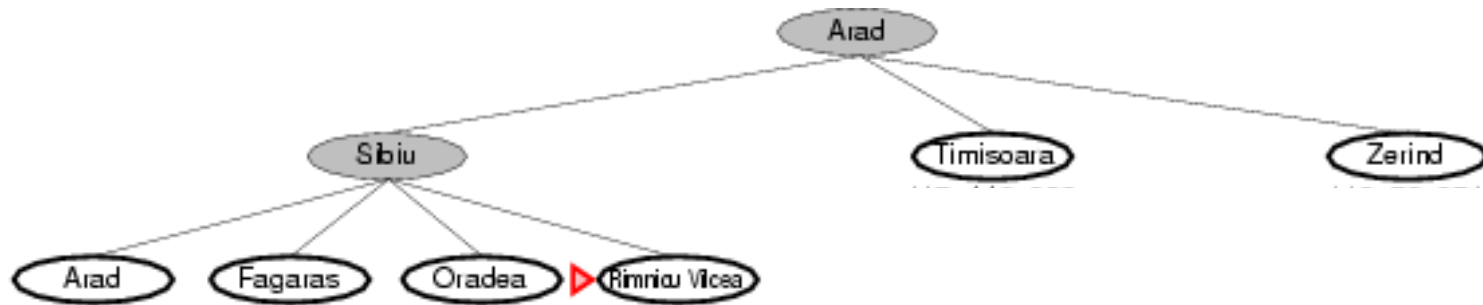


Tree Search Example



Start: Arad
Goal: Bucharest

Tree Search Example



Handling Repeated States

- Initialize the **frontier** using the **starting state**
- While the frontier is not empty
 - Choose a frontier node according to **search strategy** and take it off the frontier.
 - If the node contains the **goal state**, return solution
Else **expand** the node and add its children to the frontier.

To handle repeated states:

Every time you expand a node, add that state to the **explored set**; do not put explored states on the frontier again.

Every time you add a node to the frontier, check whether it already exists with a higher path cost, and if yes, replace that node with the new one.

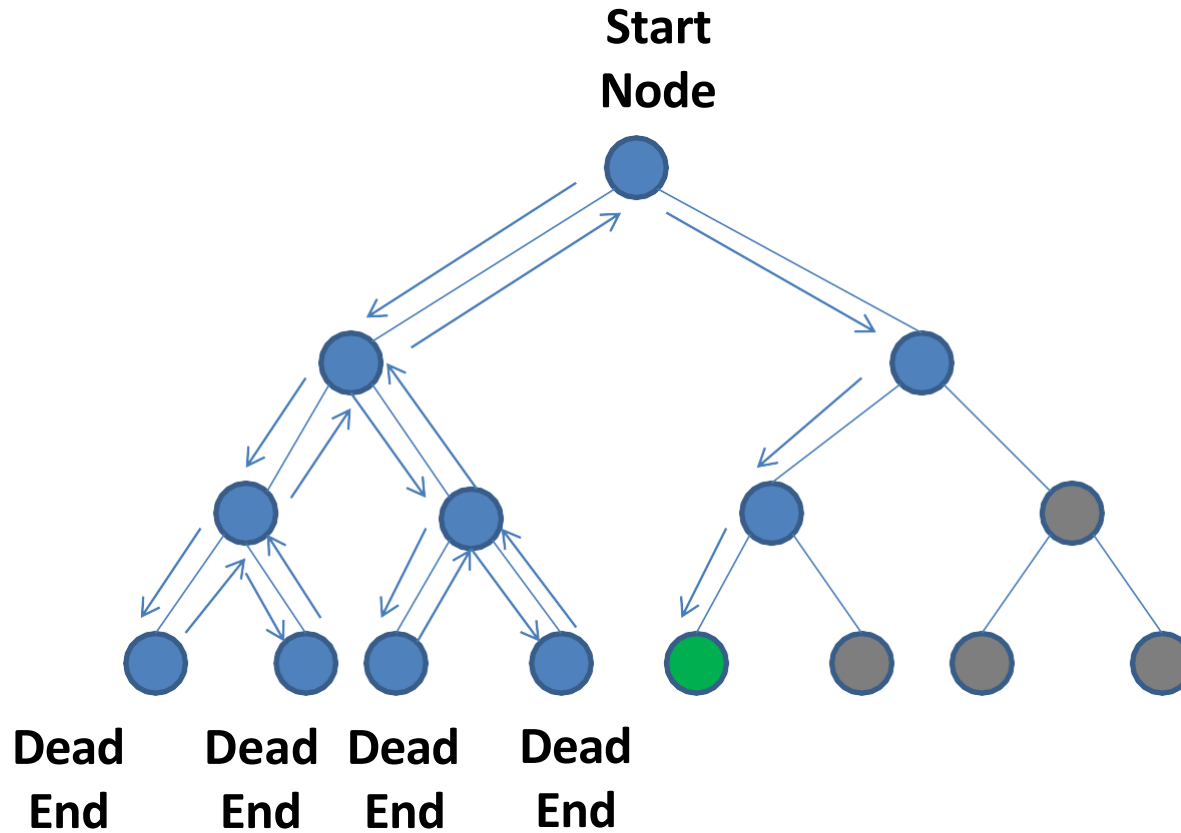
Backtracking Search

“Backtracking is a technique for systematically trying all paths through a state space”

It begins at the start state and pursues a path until:

- Finding a goal, then quit and return the solution path.
- Finding a dead end, then backtrack to the most recent unexamined node and continue down one of its branches.

Backtracking Search



Backtracking Algorithm Data Structures

- State List (SL): lists the states in the current path being tried. If the goal is found, then it contains the solution path.
- New State List (NSL): contains nodes a waiting evaluation.
- Dead Ends (DE): lists states whose descendants have failed to contain a goal node.
- Current State (CS): a state currently under consideration.

the Backtracking Algorithm

Function Backtrack;

Begin

SL := [Start]; NSL := [Start]; DE := Start;

while NSL# [] do

begin

if CS = goal (or meets goal description) then return (SL);

if CS has no children (excluding nodes already on DE, SL, and NSL)

then begin

while SL is not empty and CS = the first element of SL do

begin

add CS to DE;

remove first element from SL;

remove first element from NSL;

CS := first element of NLS;

end;

add CS to SL;

end

else begin

place children of CS on NSL; (except nodes on DE, SL, or NSL)

CS := first element of NSL;

add CS to SL;

end;

end;

return FAIL;

end;

Exercises

What have we learned?

- The following are two problems that can be solved using state-space search techniques. You should:
 - Suggest a suitable representation for the problem state.
 - State what the initial and final states are in this representation.
 - State the available operators/rules for getting from one state to the next, giving any conditions on when they may be applied.
 - Draw the first two levels of the directed state-space graph for the given problem.

Exercises

What have we learned?

- **[Problem # 1]** The Cannibals and Missionaries problem: "Three cannibals and three missionaries come to a crocodile infested river. There is a boat on their side that can be used by either one or two persons. If cannibals outnumber the missionaries at any time, the cannibals eat the missionaries. How can they use the boat to cross the river so that all missionaries survive?" Formalize the problem in terms of state-space search.
- **[Problem # 2]** A farmer with his dog, rabbit and lettuce come to the east side of a river they wish to cross. There is a boat at the river's edge, but of course only the farmer can row. The boat can only hold two things (including the rower) at any one time. If the dog is ever left alone with the rabbit, the dog will eat it. Similarly, if the rabbit is ever left alone with the lettuce, the rabbit will eat it. How can the farmer get across the river so that all four characters arrive safely on the other side?

Additional Resources

Up Next ..
Section 3.4
(video lesson)

- Backtracking (Solving a Maze – C++)

https://www.youtube.com/watch?v=gBC_Fd8EE8A

- N Queen Problem | Backtracking:

<https://www.youtube.com/watch?v=0DeznFqrgAI>

- Sudoku (Visualisation) | Backtracking:

https://www.youtube.com/watch?v=_vWRZiDUGHU

Thank you!